

Tema 01_2. Estructuras de control

Índice

- 2.1 Expresiones y sentencias**
- 2.2 Ramificaciones**
- 2.3 Booleanos**
- 2.4 Comparaciones y pruebas de igualdad**
- 2.5 Comparaciones mixtas**
- 2.6 Operadores booleanos**
- 2.7 La instrucción switch**
- 2.8 Bucles while y do**
- 2.9 Bucles for**
 - 2.9.1 El bucle for clásico**
 - 2.9.2 El bucle for of**
 - 2.9.3 El bucle for in**
- 2.10 Interrupción y continuación**
- 2.11 Captura de excepciones**

En este tema aprenderás las estructuras de control del lenguaje JavaScript: ramificaciones, bucles y captura de excepciones. El tema también ofrece una visión general de las sentencias JavaScript y describe el proceso de inserción automática de punto y coma.

2.1 Expresiones y sentencias

JavaScript, al igual que Java y C++, distingue entre expresiones y sentencias. Una expresión tiene un valor. Por ejemplo, `6 * 7` es una expresión con valor 42. Una llamada a un método como `Math.max(6, 7)` es otro ejemplo de expresión.

Una instrucción nunca tiene un valor. En cambio, se ejecuta para lograr algún efecto.

Por ejemplo,

```
let number = 6 * 7;
```

es una instrucción cuyo efecto es declarar e inicializar la variable `number`. Este tipo de instrucción se denomina declaración de variable.

Aparte de las declaraciones de variables, otros tipos de sentencias comunes son las ramificaciones y los bucles. Los verás más adelante en este tema.

La forma más simple de una instrucción es una *instrucción de expresión*. Consiste en una expresión seguida de un punto y coma. Aquí tienes un ejemplo:

```
console.log(6 * 7);
```

La expresión `console.log(6 * 7)` tiene un efecto secundario: muestra 42 en la consola. También tiene un valor, que resulta ser indefinido, ya que el método `console.log` ha optado por no devolver nada más interesante. Incluso si la expresión tuviera un valor más interesante, no importaría, ya que el valor de una instrucción de expresión se descarta.

Por lo tanto, una instrucción de expresión solo es útil para una expresión que tiene un efecto secundario. La instrucción de expresión

```
6 * 7;
```

es válida en JavaScript, pero no tiene ningún efecto en un programa.

Es útil comprender la diferencia entre expresiones y sentencias, pero en JavaScript es un poco complicado ver la diferencia entre una expresión y una sentencia de expresión. Como verás en la siguiente sección, se añade automáticamente un punto y coma si escribes una línea que contiene una sola expresión, convirtiéndola en una sentencia. Por esa razón, no puedes observar una expresión en la consola JavaScript de un navegador o en Node.js.

Por ejemplo, prueba a escribir `6 * 7`. Se muestra el valor de la expresión:

```
6 * 7
```

```
42
```

Eso es lo que hace un bucle de lectura-evaluación-impresión, o REPL: lee una expresión, la evalúa e imprime el valor.

Sin embargo, debido a la inserción automática del punto y coma, el REPL de JavaScript ve en realidad la *instrucción*

```
6 * 7;
```

Las sentencias no tienen valores, pero el REPL de JavaScript muestra valores para ellas de todos modos.

Intente escribir una declaración de variable:

```
let number = 6 * 7;
```

```
undefined
```

Como acabas de ver, para una instrucción de expresión, el REPL muestra el valor de la expresión. Para una declaración de variable, el REPL muestra `undefined`.

Cuando realice sus propios experimentos con el REPL, es importante que sepa cómo interpretar el resultado. Por ejemplo, escriba esta instrucción de expresión y observe la respuesta:

```
console.log(6 * 7);
```

```
42
```

```
undefined
```

La primera línea del resultado es el efecto secundario de la llamada a `console.log`. La segunda línea es el valor de retorno de la llamada al método. Como ya se ha mencionado, el método `console.log` devuelve `undefined`.

2.2 Ramificaciones

La instrucción condicional en JavaScript tiene la forma

```
if (condición) instrucción
```

La condición debe estar entre paréntesis.



Consejo

En la condición, debe producir verdadero o falso, aunque JavaScript permite valores arbitrarios y los convierte a valores booleanos. Como verá en la siguiente sección, estas conversiones pueden ser poco intuitivas y potencialmente peligrosas. Siga la regla de oro n.º 3 del prefacio:

- Conoce tus tipos y evita la conversión automática de tipos.

A menudo querrás ejecutar varias sentencias cuando se cumpla una condición. En este caso, utiliza una *sentencia de bloque* que adopte la forma

```
{  
  instrucción1  
  instrucción2  
  . . .  
}
```

Una cláusula `else` opcional se ejecuta cuando no se cumple la condición, por ejemplo:

```
if (sus ventas > objetivo) {  
  rendimiento = 'Bueno'  
  bonus = 100  
} else {  
  rendimiento = 'Mediocre'  
  bonificación = 0  
}
```



Nota

Este ejemplo muestra el «estilo de llaves único», en el que la llave de apertura se coloca al final de la línea que precede a la primera instrucción del bloque. Este estilo se utiliza habitualmente con JavaScript.

Si la cláusula `else` es otra instrucción `if`, se utiliza convencionalmente el siguiente formato:

```
if (yourSales > 2 * target) {  
  performance = 'Excelente'  
  bonus = 1000  
} else if (yourSales > target {  
  rendimiento = 'Bueno'  
  bonificación = 100  
} else {  
  rendimiento = 'Medio'  
  bonificación = 0  
}  
No es necesario utilizar llaves en las sentencias simples:  
if (yourSales > target  
  bonificación = 100
```



Precaución

Si no utilizas llaves, o si las utilizas, pero no sigues el «estilo de llaves único» con una instrucción if/else, puedes escribir código que funcione en un archivo de programa pero que falle al pegarlo en una consola JavaScript. Considera este ejemplo:

```
if (tusVentas > objetivo)
  bonus = 100
else
  bonus = 0
```

Algunas consolas JavaScript analizan el código línea por línea. Una consola de este tipo considerará que la instrucción if está completa antes de la cláusula else. Para evitar este problema, utilice llaves o coloque toda la instrucción if en una sola línea

```
if (tusVentas > objetivo) bonificación = 100; else bonificación = 0
```

A veces resulta conveniente tener una expresión análoga a la instrucción if. Considere el cálculo del mayor de dos valores:

```
let max = undefined
if (x > y) max = x; else max = y
```

Sería mejor inicializar max con el mayor de x e y. Dado que if es una instrucción, no podemos escribir:

```
let max = if (x > y) x else y // Error: no se espera una instrucción if
```

En su lugar, utilice el operador **?** :, también llamado operador «condicional». La expresión *condición ? primero : segundo* evalúa **primero** si se cumple la condición, y **segundo** en caso contrario. Esto resuelve nuestro problema:

```
let max = x > y ? x : y
```



Nota

La expresión

```
x > y ? x : y
```

es un ejemplo práctico para ilustrar el operador condicional, pero debe utilizar el método de la biblioteca estándar Math.max si necesita el mayor de dos o más valores.

2.3 Booleanos

Esta es una sección «descabellada» que describe con cierto detalle una característica confusa de JavaScript.

En JavaScript, las condiciones (como la de la instrucción if) no tienen por qué ser valores booleanos. Los valores «falsos» 0, NaN, null, undefined y la cadena vacía hacen que la condición falle. Todos los demás valores son «verdaderos» y hacen que la condición tenga éxito. A menudo también se denominan «falsos» o «verdaderos». Ninguno de estos es un término oficial en la especificación del lenguaje.



Nota

La *booleidad* también se aplica a las condiciones de bucle, los operandos de los operadores booleanos `&&`, `||` y `!`, y el primer operando del operador `?` :. Todas estas construcciones se tratan más adelante en este tema.

La regla de conversión booleana parece razonable a primera vista. Supongamos que tienes una variable `performance` y solo quieres usarla si no está indefinida. Entonces escribes:

```
if (performance) . . . // Peligro
```

Por supuesto, la prueba falla como era de esperar si `performance` no está definido. Como extra, también falla si `performance` es nulo.

Pero ¿qué pasa si `performance` es una cadena vacía? ¿O el número cero? ¿Realmente quieres tratar estos valores de la misma manera que los valores ausentes? A veces sí, y a veces no. ¿No debería tu código indicar claramente cuál es tu intención? Simplemente escribe lo que quieres decir:

```
if (performance !== undefined) . . .
```

2.4 Pruebas de comparación e igualdad

JavaScript tiene la variedad habitual de operadores de comparación:

```
<  menor que
<= menor o igual
>  mayor que
>= mayor o igual
```

Cuando se utilizan para comparar números, estos operadores no presentan ninguna sorpresa:

```
3 < 4 // verdadero
3 >= 4 // falso
```

Cualquier comparación que implique NaN da como resultado falso:

```
NaN < 4 // falso
NaN >= 4 // falso
NaN <= NaN // falso
```

Los mismos operadores también comparan cadenas, utilizando el orden lexicográfico.

```
«Hola» < «Adiós» // falso: la H viene después de la G
«Hello» < «Hi» // verdadero: la e viene antes de la i
```

Al comparar valores con

```
<, <=, >, >=
```

asegúrese de que ambos operandos sean números o ambos operandos sean cadenas. Convierta los operandos explícitamente si es necesario. De lo contrario, JavaScript convertirá los operandos por usted, a veces con resultados indeseables; consulte la siguiente sección.

Utilice estos operadores para comprobar la igualdad:

```
=== estrictamente igual a
!== no estrictamente igual a
```

Los operadores de igualdad estricta son sencillos. Los operandos de diferentes tipos nunca son estrictamente iguales. Los valores indefinidos y nulos solo son estrictamente iguales a sí mismos. Los números, los valores booleanos y las cadenas son estrictamente iguales si sus valores son iguales.

```
'42' === 42 // falso: tipos diferentes
indefinido === nulo // falso
'42' === '4' + 2 // verdadero: mismo valor de cadena '42'
```

También hay operadores de «igualdad aproximada» `==` y `!=` que pueden comparar valores de diferentes tipos. Por lo general, esto no resulta útil; consulte la siguiente sección si le interesan los detalles.



Precaución

No se puede utilizar

```
x === NaN
```

para comprobar si `x` es igual a `NaN`. No hay dos valores `NaN` que se consideren iguales entre sí. En su lugar, llame a `Number.isNaN(x)`.



Nota

`Object.is(x, y)` es casi igual que `x === y`, excepto que `Object.is(+0, -0)` es falso y `Object.is(NaN, NaN)` es verdadero.

Al igual que en Java y Python, la igualdad de objetos (incluidas las matrices) significa que los dos operandos hacen referencia al mismo objeto. Las referencias a objetos diferentes nunca son iguales, incluso si ambos objetos tienen el mismo contenido.

```
let harry = { name: 'Harry Smith', age: 42 }
let harry2 = harry
harry === harry2 // verdadero: dos referencias al mismo objeto
let harry3 = { name: 'Harry Smith', age: 42 }
harry === harry3 // falso: objetos diferentes
```

2.5 Comparaciones mixtas

Esta es otra sección «loca» que describe con cierto detalle características de JavaScript que pueden resultar confusas. Por supuesto, omita esta sección si sigue el consejo de la regla de oro n.º 3: evite las comparaciones de tipos mixtos y, en particular, los operadores de «igualdad débil» (`==` y `!=`).

¿Sigues aquí? Veamos primero las comparaciones de tipos mixtos con los operadores

```
<, <=, > y >=
```

Si uno de los operandos es un número, el otro operando se convierte en un número. Supongamos que el otro operando es una cadena. La conversión produce el valor numérico si la cadena contiene un número, 0 si la cadena está vacía o `NaN` en caso contrario. Además, cualquier comparación que implique `NaN` es falsa, incluso `NaN <= NaN`.

```
'42' < 5 // falso: '42' se convierte en el número 42
'' < 5 // verdadero: '' se convierte en el número 0
'Hola' <= 5 // falso: «Hola» se convierte en NaN
5 <= 'Hello' // falso: «Hello» se convierte en NaN
```

Ahora supongamos que el otro operando es una matriz:

```
[4] < 5 // verdadero: [4] se convierte en el número 4  
[] < 5 // verdadero: [] se convierte en el número 0  
[3, 4] < 5 // falso: [3, 4] se convierte en NaN
```

Si ninguno de los operandos es un número, ambos se convierten en cadenas. Estas comparaciones rara vez producen resultados significativos:

```
[1, 2, 3] < {} // verdadero: [1, 2, 3] se convierte en «1,2,3» y {} en «[object Object]»
```

A continuación, veamos más detenidamente la igualdad aproximada `x == y`. Así es como funciona:

- Si los dos operandos son del mismo tipo, compárelos estrictamente.
- Los valores `undefined` y `null` son aproximadamente iguales a sí mismos *y entre sí*, pero no a ningún otro valor.
- Si un operando es un número y el otro una cadena, se convierte la cadena en un número y se comparan estrictamente.
- Si uno de los operandos es un valor booleano, convierta ambos a números y compárelos estrictamente.
- Si un operando es un objeto y el otro no, convierte el objeto a un tipo primitivo y, a continuación, compáralos de forma aproximada.

Por ejemplo:

```
'' == 0 // verdadero: '' se convierte en 0  
'0' == 0 // verdadero: '0' se convierte en 0  
'0' == false // verdadero: ambos se convierten en 0  
undefined == false // falso: undefined solo es igual a sí mismo y a null
```

Vuelve a fijarte en las cadenas `''` y `'0'`. Ambas son «iguales a» `0`. Pero no son «iguales entre sí»:

```
'' == '0' // falso: no hay conversión, ya que ambos operandos son cadenas
```

Como puede ver, las reglas de comparación aproximadas no son muy útiles y pueden dar lugar fácilmente a errores sutiles. Evite este problema utilizando operadores de igualdad estricta (`===` y `!==`).



Nota

La comparación aproximada `x == null` comprueba en realidad si `x` es indefinido o nulo, y `x != null` comprueba si `x` no es ninguno de los dos. Algunos programadores que han decidido no utilizar nunca la igualdad aproximada hacen una excepción en este caso.

2.6 Operadores booleanos

JavaScript tiene tres operadores para combinar valores booleanos:

```
&& y  
|| o  
! not
```

La expresión **x && y** es verdadera si tanto **x** como **y** son verdaderas, y **x || y** es verdadera si al menos una de **x** e **y** lo es. La expresión **!x** es verdadera si **x** es falsa.

Los operadores **&&** y **||** se evalúan de forma perezosa. Si el operando izquierdo decide el resultado (falso para **&&**, verdadero para **||**), el operando derecho no se evalúa. Esto suele ser útil, por ejemplo:

```
if (i < a.length && a[i] > 0) // a[i] > 0 no se evalúa si i ≥ a.length
```

Los operandos **&&** y **||** tienen otra curiosidad si los operandos no son valores booleanos. Proporcionan uno de los operandos como valor de la expresión. Si el operando izquierdo determina el resultado, se convierte en el valor de la expresión y el operando derecho no se evalúa. De lo contrario, el valor de la expresión es el valor del operando derecho.

Por ejemplo:

```
0 && 'Harry' // 0
0 || 'Harry' // 'Harry'
```

Algunos programadores intentan aprovechar este comportamiento y escriben código como el siguiente:

```
let result = arg && arg.someMethod()
```

La intención es comprobar que **arg** no sea indefinido o nulo antes de llamar al método. Si lo es, entonces **result** también es indefinido o nulo. Esta expresión falla si **arg** es cero, una cadena vacía o falso.

Otro uso es producir un valor predeterminado cuando un método devuelve indefinido o nulo:

```
let resultado = arg.someMethod() || valorPredeterminado
```

Una vez más, esto falla si el método puede devolver cero, una cadena vacía o falso.

La expresión **x ?? y** devuelve **x** si **x** no es indefinido o nulo, e **y** en caso contrario. En la expresión

```
let result = arg.someMethod() ?? defaultValue
```

el valor predeterminado se utiliza solo cuando el método devuelve indefinido o nulo.

La expresión **x?.propertyName** devuelve la propiedad dada si **x** no es indefinido o nulo, e indefinido en caso contrario. Considere

```
let recipient = person?.name
```

Si **person** no es indefinido ni nulo, entonces el lado derecho es exactamente igual a **person.name**. Pero si **person** es indefinido o nulo, entonces **recipient** se establece como indefinido. Si hubiera utilizado el operador **.** en lugar de **?.**, se habría producido una excepción.

Puedes encadenar los operadores **?.**

```
let recipientLength = person?.name?.length
```

Si **person** o **person.name** es indefinido o nulo, entonces **recipientLength** se establece como indefinido.



Precaución

Algunos programadores utilizan la expresión $x \mid 0$ para eliminar la parte fraccionaria de un número x . Esto produce resultados incorrectos si $x \geq 231$. Es mejor utilizar `Math.floor(x)` en su lugar.

2.7 La instrucción `switch`

JavaScript tiene una instrucción `switch` que es igual que la instrucción `switch` en C, C++, Java y C#, con todos sus defectos.

La instrucción `switch` compara una expresión con muchos valores posibles. A continuación, se muestra un ejemplo:

```
let description = ''
switch (someExpression) {
  case 0:
    description = 'zero'
    break
  case false:
  case true:
    description = 'boolean'
    break
  case '':
    description = 'empty string'
  default:
    description = 'something else'
}
```

La ejecución comienza en la etiqueta de caso que es estrictamente igual al valor de la expresión y continúa hasta el siguiente salto o el final de la instrucción `switch`. Si ninguna de las etiquetas de caso coincide, la ejecución comienza en la etiqueta predeterminada, si existe.

Dado que se utiliza la igualdad estricta para la coincidencia, las etiquetas de caso no deben ser objetos.



Precaución

Si olvida añadir un `break` al final de una alternativa, la ejecución pasa a la siguiente alternativa. Esto ocurre en el ejemplo anterior cuando el valor es una cadena vacía. La descripción se establece primero en «cadena vacía» y luego en «otra cosa». Este comportamiento de «paso» es claramente peligroso y una causa común de errores. Por esa razón, algunos desarrolladores evitan la instrucción `switch`.



Consejo

En muchos casos, la diferencia de rendimiento entre una instrucción `switch` y el conjunto equivalente de instrucciones `if` es insignificante. Sin embargo, si tienes un gran número de casos, la máquina virtual puede utilizar una «tabla de salto» para saltar de forma eficiente al caso adecuado.

2.8 Bucles `while` y `do`

El bucle `while` ejecuta una instrucción (que puede ser una instrucción de bloque) mientras se cumple una condición. La forma general es

```
while (condición) instrucción
```

El siguiente bucle determina cuánto tiempo tardará en ahorrar una cantidad específica de dinero para su merecida jubilación, suponiendo que deposite la misma cantidad de dinero cada año y que el dinero genere un tipo de interés específico.

```
let years = 0
while (balance < goal) {
  balance += paymentAmount
  let interest = balance * interestRate / 100
  balance += interest
  years++
}
console.log(`${years} years.`)
```

El bucle while nunca se ejecutará si la condición es falsa desde el principio. Si quieres asegurarte de que un bloque se ejecute al menos una vez, debes mover la prueba al final, utilizando el bucle do/while. Su sintaxis es la siguiente:

```
do statement while (condición)
```

Este bucle ejecuta la instrucción (que suele ser un bloque) y luego comprueba la condición. Si se cumple la condición, se repiten la instrucción y la prueba. Veamos un ejemplo. Supongamos que acabamos de procesar s[i] y ahora estamos buscando el siguiente espacio en la cadena:

```
do {
  i++
} while (i < s.length && s[i] !== ' ')
```

Cuando finaliza el bucle, o bien i ha superado el final de la cadena, o bien s[i] es un espacio.

El bucle do es mucho menos común que el bucle while.

2.9 Bucles for

El bucle for es una construcción general para iterar sobre elementos. Las tres secciones siguientes tratan las variantes que ofrece JavaScript.

2.9.1 El bucle for clásico

La forma clásica del bucle for funciona igual que en C, C++, Java o C#. Funciona con un contador o una variable similar que se actualiza después de cada iteración. El siguiente bucle registra los números del 1 al 10:

```
for (let i = 1; i <= 10; i++)
  console.log(i)
```

La primera ranura de la instrucción for contiene la inicialización del contador. La segunda ranura proporciona la condición que se comprobará antes de cada nueva pasada por el bucle. La tercera ranura especifica cómo actualizar el contador después de cada iteración del bucle.

La naturaleza de la inicialización, la prueba y la actualización depende del tipo de recorrido que desee. Por ejemplo, este bucle recorre los elementos de una matriz en orden inverso:

```
for (let i = a.length - 1; i >= 0; i--)
  console.log(a[i])
```



Consejo

Puede colocar declaraciones o expresiones variables arbitrarias en la primera ranura y expresiones arbitrarias en las otras ranuras de un bucle `for`. Sin embargo, es una regla no escrita de buen gusto que se debe inicializar, probar y actualizar la misma variable.



Nota

Es posible introducir varias expresiones de actualización en la tercera ranura de un bucle `for` utilizando el *operador coma*:

```
for (let i = 0, j = a.length - 1; i < j; i++, j--) {  
  let temp = a[i]  
  a[i] = a[j]  
  a[j] = temp  
}
```

En la expresión `i++, j--`, el operador coma une las dos expresiones `i++` y `j--` en una nueva expresión. El valor de una expresión coma es el valor del segundo operando. En esta situación, el valor no se utiliza, solo nos interesan los efectos secundarios del incremento y el decremento.

El operador coma no suele gustar porque puede resultar confuso. Por ejemplo, `Math.max((9, 3))` es el máximo de los valores únicos (9, 3), es decir, 3.

La coma en la declaración `let i = 0, j = a.length - 1` no es un operador coma, sino una parte sintáctica de la instrucción `let`. Esta instrucción declara dos variables, `i` y `j`.

2.9.2 El bucle `for of`

El bucle `for of` itera sobre los elementos de un *objeto iterable*, normalmente una matriz o una cadena.

Aquí hay un ejemplo:

```
let arr = [, 2, , 4]  
arr[9] = 100  
for (const element of arr)  
  console.log(element) // Imprime undefined, 2, undefined, 4, undefined (5 veces), 100
```

El bucle recorre todos los elementos de la matriz desde el índice 0 hasta `arr.length - 1`, en orden ascendente. Los elementos de los índices 0, 2 y 4 a 8 se reportan como indefinidos.

La variable `element` se crea en cada iteración del bucle y se inicializa con el valor del elemento actual. Se declara como `const`, ya que no se modifica en el cuerpo del bucle.

El bucle `for of` es una mejora agradable con respecto al bucle `for` clásico si necesitas procesar todos los elementos de una matriz. Sin embargo, todavía hay muchas oportunidades para usar el bucle `for` clásico. Por ejemplo, es posible que no quieras recorrer toda la matriz o que necesites el valor del índice dentro del bucle.

Cuando el bucle `for of` itera sobre una cadena, visita cada *punto de código Unicode*. Ese es el comportamiento que se desea. Por ejemplo:

```
let greeting = 'Hello 🌐 '  
for (const c of greeting)  
  console.log(c) // Imprime Hello, un espacio y 🌐
```

No debe preocuparse por el hecho de que 🌐 utilice dos unidades de código, almacenadas en `greeting[6]` y `greeting[7]`.

2.9.3 El bucle `for in`

No se puede utilizar el bucle **for of** para iterar sobre los valores de propiedad de un objeto arbitrario, y probablemente no querría hacerlo, ya que los valores de propiedad suelen carecer de sentido sin las claves. En su lugar, visite las claves con el bucle **for in**:

```
let obj = { name: 'Harry Smith', age: 42 }
for (const key in obj)
  console.log(`${key}: ${obj[key]}`)
```

Este bucle imprime `age: 42` y `name: Harry Smith` en algún orden.

El bucle **for in** recorre las claves del objeto dado. Las propiedades «prototipo» se incluyen en la iteración, mientras que ciertas propiedades «no enumerables» se omiten. El orden en que se recorren las claves depende de la implementación, por lo que no debes confiar en él.

Puede utilizar un bucle **for in** para iterar sobre los nombres de propiedad de una matriz.

```
let numbers = [1, 2, , 4]
numbers[99] = 100
for (const i in numbers)
  console.log(`${i}: ${numbers[i]}`)
```

Este bucle establece `i` en «0», «1», «3» y «99». Ten en cuenta que, al igual que todos los objetos JavaScript, las claves de las propiedades son cadenas. Aunque las implementaciones comunes de JavaScript iteran sobre las matrices en orden numérico, es mejor no confiar en ello. Si el orden de iteración es importante para ti, lo mejor es utilizar un bucle **for of** o un bucle **for** clásico.



Precaución

Ten cuidado con expresiones como `numbers[i + 1]` en un bucle **for in**. Por ejemplo,

```
if (numbers[i] === numbers[i + 1]) // ¡Error! i + 1 es «01», «11», y así sucesivamente
```

La condición *no* compara elementos adyacentes. Dado que `i` contiene una cadena, el operador `+` *concatena* cadenas. Si `i` es «0», entonces `i + 1` es «01».

Para solucionar este problema, convierte la cadena `i` en un número:

```
if (numbers[i] === numbers[parseInt(i) + 1])
```

O utilice un bucle `for` clásico.

Por supuesto, si añades otras propiedades a tu matriz, también se visitarán:

```
numbers.lucky = true
for (const i in numbers) // i es '0', '1', '3', '99', 'lucky'
  console.log(`${i}: ${numbers[i]}`)
```

Es posible que otros añadan propiedades enumerables a `Array.prototype` u `Object.prototype`. Estas aparecerán en un bucle **for in**. Por lo tanto, las normas modernas de JavaScript desaconsejan encarecidamente esta práctica. No obstante, algunos programadores advierten contra el bucle **for in**

porque les preocupan las bibliotecas heredadas o los compañeros que pegan código aleatorio de Internet.



Nota

En el siguiente tema, aprenderás otra forma de iterar sobre un array, utilizando técnicas de programación funcional. Por ejemplo, puedes registrar todos los elementos del array de esta manera:

```
arr.forEach((element, key) => { console.log(`${key}: ${element}`) })
```

La función proporcionada se llama para todos los elementos y claves de índice (como números 0 1 3 99, no cadenas).



Precaución

Cuando el bucle **for in** itera sobre una cadena, visita los índices de cada *unidad de código Unicode*. Probablemente eso no sea lo que usted desea. Por ejemplo:

```
let greeting = 'Hello 🌐'
for (const i of greeting)
  console.log(greeting[i])
  // Imprime Hello, un espacio y dos símbolos rotos
```

Los índices 6 y 7 de las dos unidades de código del carácter Unicode 🌐 se visitan por separado.

2.10 Interrupción y continuación

A veces, se desea salir de un bucle tan pronto como se alcanza un objetivo. Supongamos que se busca la posición del primer elemento negativo en una matriz:

```
let i = 0
while (i < arr.length) {
  if (arr[i] < 0) . . .
  . . .
}
```

Al ver un elemento negativo, solo quieres salir del bucle, de modo que *i* permanezca en la posición del elemento. Eso es lo que logra la instrucción `break`.

```
let i = 0
while (i < arr.length) {
  if (arr[i] < 0) break
  i++
}
// Llegar aquí después de break o cuando el bucle termine normalmente
```

La instrucción `break` nunca es necesaria. Siempre se puede añadir una variable booleana para controlar la finalización del bucle, a menudo denominada algo así como `done` o `found`:

```
let i = 0
let found = false
while (!found && i < arr.length) {
  if (arr[i] < 0) {
    found = true
  } else {
    i++
  }
}
```

```
}  
}
```

Al igual que Java, JavaScript ofrece una instrucción *break etiquetada* que permite salir de múltiples bucles anidados. Supongamos que desea encontrar la ubicación del primer elemento negativo en una matriz bidimensional. Cuando lo haya encontrado, deberá salir de dos bucles. Añada una etiqueta (es decir, un identificador seguido de dos puntos) *antes* del bucle exterior. Un *break* etiquetado salta *después* del bucle etiquetado:

```
let i = 0  
let j = 0  
outer:  
while (i < arr.length) {  
  while (j < arr[i].length) {  
    if (arr[i][j] < 0) break outer  
    j++  
  }  
  i++  
  j = 0  
}  
// Llegar aquí después de break outer o cuando ambos bucles terminen normalmente
```

La etiqueta en una instrucción *break* etiquetada debe estar en la misma línea que la palabra clave *break*.

Los *breaks* etiquetados no son comunes.

Por último, existe una instrucción *continue* que, al igual que la instrucción *break*, interrumpe el flujo normal de control. La instrucción *continue* transfiere el control al final del bucle más interno que lo envuelve. A continuación, se muestra un ejemplo: calcular la media de los elementos positivos de una matriz:

```
let count = 0  
let sum = 0  
for (let i = 0; i < arr.length; i++) {  
  if (arr[i] <= 0) continue  
  count++  
  sum += arr[i]  
}  
let avg = count === 0 ? 0 : sum / count
```

Cuando un elemento no es positivo, la instrucción *continue* salta inmediatamente al encabezado del bucle, omitiendo *solo* el resto de *la* iteración *actual*.

Si se utiliza una instrucción *continue* en un bucle *for*, salta a la parte de «actualización» del bucle *for*, como en este ejemplo.

También existe una forma etiquetada de la instrucción *continue* que salta al final del bucle con la etiqueta correspondiente. Estas instrucciones son muy poco comunes.

Muchos programadores encuentran confusas las sentencias *break* y *continue*. Son fáciles de evitar.

2.11 Captura de excepciones

Algunos métodos devuelven un valor de error cuando se invocan con argumentos no válidos. Por ejemplo, `parseFloat('')` devuelve un valor NaN.

Sin embargo, no siempre es buena idea devolver un valor de error. Puede que no haya una forma obvia de distinguir entre valores válidos e inválidos. El método `parseFloat` es un buen ejemplo. La llamada

`parseFloat('NaN')` devuelve NaN, al igual que `parseFloat('Infinity')` devuelve el valor Infinity. Cuando `parseFloat` devuelve NaN, no se puede saber si ha analizado una cadena «NaN» válida o un argumento inválido.

En JavaScript, un método puede tomar una ruta de salida alternativa si no puede completar su tarea de la forma habitual. En lugar de devolver un valor, un método puede *lanzar una excepción*. En ese caso, la ejecución no se reanuda en el código que llamó al método. En su lugar, se ejecuta una *cláusula catch*. Si no se captura ninguna excepción, el programa finaliza.

Para capturar una excepción, utilice una instrucción `try`. La forma más simple de esta instrucción es la siguiente:

```
try {  
  código  
  más código  
  más código  
} catch {  
  controlador  
}
```

Si cualquier código dentro del bloque `try` lanza una excepción, el programa omite el resto del código del bloque `try` y ejecuta el código del controlador dentro de la cláusula `catch`.

Por ejemplo, supongamos que recibe una cadena JSON y la analiza. La llamada a `JSON.parse` lanza una excepción si el argumento no es JSON válido. Maneje esa situación en la cláusula `catch`:

```
let input = . . . // Leer la entrada desde algún lugar  
try {  
  let data = JSON.parse(input)  
  // Si la ejecución continúa aquí, la entrada es válida  
  // Procesar datos  
  . . .  
} catch {  
  // Tratar el hecho de que la entrada no es válida  
  . . .  
}
```

En el controlador, puede registrar esa información o tomar alguna medida evasiva para tratar el hecho de que se le ha entregado una cadena JSON incorrecta.